

RideNow — Master Architecture & Interview Preparation Guide

Comprehensive End-to-End System Walkthrough & 30+ Technical Interview Scenarios
Explained Simply

💡 HOW TO USE THIS DOCUMENT

This guide is designed for two purposes: (1) To teach you **every single concept, algorithm, and file** of the RideNow codebase in plain, intuitive language, and (2) To prepare you to answer **any system design, coding, or architecture interview question** with complete confidence.

📖 Chapter 1: The Big Picture (Explained Simply)

Imagine you want to travel from your college to a coffee shop. You open your phone, tap "Book Auto", and in 30 seconds, a nearby driver accepts your trip, drives to your location, picks you up, drops you off, and you pay ₹60 in Cash or UPI.

RideNow is the complete full-stack software brain that makes this entire experience happen seamlessly in real-time. Just like Rapido, Uber, or Ola, RideNow connects three distinct groups of users:

User Role	Who They Are	What They Do in RideNow
Passenger (Customer)	Everyday riders commuting across the city	Selects pickup & dropoff on an interactive map, views fares in Indian Rupees (₹ INR), chooses Cash/Online, and tracks the captain live.
Captain (Driver)	Bike, Auto, and Cab vehicle owners	Toggles Duty (ONLINE/OFFLINE), sends live GPS breadcrumbs, receives instant animated ride requests, accepts/declines trips, and tracks daily earnings.

User Role	Who They Are	What They Do in RideNow
Operations Admin	RideNow central support & fleet managers	Approves driver licenses, configures per-km pricing rates, monitors live rides, resolves disputes, and audits staff actions.

Chapter 2: System Architecture & The 4 Applications

The RideNow codebase is organized into four clean, decoupled projects:

Application Directory	Tech Stack	Core Responsibility
<code>backend</code> (Port 3000)	Node.js, Express, TypeScript, Prisma ORM, PostgreSQL, Socket.IO, Stripe, Firebase	Central REST API, automated dispatch engine, payment holds/captures, JWT auth, and WebSocket event hub.
<code>customer-web</code> (Port 5173)	React 18, Vite, TypeScript, MapLibre GL, OSRM Routing	Passenger web app with address auto-complete, route drawing, ₹ INR fare simulator, and live driver tracking.
<code>driver-mobile</code> (Port 8081)	React Native, Geolocation, Socket.IO Client	Captain mobile app with duty toggle, GPS tracker, incoming request modal (Accept/Decline), and trip milestones.
<code>admin-web</code> (Port 5174)	React 18, Vite, TypeScript, Dark/Light Themes	11-module fleet control center for captain KYC verification, manual/auto-dispatch, live fare rates, and RBAC staff management.



Chapter 3: Database Schema & Entity Blueprint

RideNow uses PostgreSQL with Prisma ORM (`backend/prisma/schema.prisma`). Here is what every table stores:

- **User & AdminStaff** : Core authentication credentials (email, bcrypt password hash, role: CUSTOMER, DRIVER, ADMIN) and mobile push notification tokens (fcmToken).
- **Customer** : Passenger profile with phone number, saved Stripe customer ID, trusted emergency contacts, and payment preferences.
- **Driver , Vehicle & DriverLocation** : Captain verification details (license number, `isApproved` , `status: OFFLINE | ONLINE | BUSY`), registered vehicle (make, model, license plate, strict type: `BIKE | AUTO | CAB`), and live GPS coordinates.
- **Ride & RideStatusHistory** : Complete trip ledger (pickup/dropoff coordinates, authoritative ₹ INR fare, vehicle type, distance, duration, assigned driver ID, and status) with an immutable audit timeline.
- **RideDeclinedDriver** : Tracks drivers who declined a specific ride, isolating them so they are not re-prompted while the engine rolls over to the next captain.
- **Payment** : Financial transaction ledger (amount in ₹ INR, method: CASH | UPI | CARD, status: PENDING | AUTHORIZED | COMPLETED | FAILED | REFUNDED, provider, gateway transaction ID).
- **Pricing** : Dynamic fare rates per vehicle class (base fare, per-km rate, per-minute rate, minimum fare).

THE COMPLETE RIDE STATUS LIFECYCLE

REQUESTED → SEARCHING_DRIVER → DRIVER_ASSIGNED → DRIVER_ARRIVING →
DRIVER_ARRIVED → RIDE_STARTED → RIDE_COMPLETED (OR CANCELLED /
NO_DRIVER_AVAILABLE)

⚡ Chapter 4: Core Workflows & Algorithms

1. Authoritative Fare Estimation

When a passenger selects locations, the backend calculates the fare authoritatively:

- **Haversine Formula**: Calculates straight-line great-circle distance between coordinates in kilometers.

- **OSRM Road Geometry:** Computes realistic driving route distance and travel duration in minutes.
- **INR Formula:** $\text{Total Fare} = \text{Base Fare} + (\text{Distance in km} \times \text{Per Km Rate}) + (\text{Duration in mins} \times \text{Per Minute Rate})$.
- **Security Rule:** The frontend never determines the price. The backend calculates and validates all pricing against the `Pricing` table.

2. The Automatic Dispatch & Driver Matching Engine

When a ride is ready for matching (Cash booking or Online payment hold):

1. **Strict 5-Point Eligibility Check:** Captain must be (1) `isApproved === true`, (2) `isSuspended === false`, (3) `status === ONLINE`, (4) `vehicle.type === ride.vehicleType` (Strict matching: Bike → Bike, Auto → Auto, Cab → Cab), and (5) Captain has no ongoing active trip.
2. **Proximity Sorting:** Filters out any driver who already declined this ride, and sorts remaining eligible drivers by distance to pickup (closest captain first).
3. **Targeted Alert:** Sends an instant WebSocket event (`incoming_ride_request`) to the closest matching captains and dispatches an FCM push notification.
4. **Automatic Roll-Over:** If the driver declines, the decline is saved in `RideDeclinedDriver`, and the engine immediately dispatches to the next candidate.

3. Atomic Concurrency Lock (Preventing Double-Booking)

To ensure two drivers tapping "Accept" at the exact same millisecond never both get assigned, RideNow executes an atomic transaction with conditional write locking:

DATABASE CONCURRENCY LOGIC

```
updateMany({ where: { id: rideId, status: 'SEARCHING_DRIVER' }, data: {
  status: 'DRIVER_ASSIGNED', driverId: winningDriverId } })
```

PostgreSQL serializes the write. The first driver updates 1 row and succeeds (HTTP 200). The second driver matches 0 rows, throws

`CONCURRENT_MODIFICATION_DETECTED`, and receives HTTP 409 Conflict.

Chapter 5: Master Interview Questions & Answers (30+ Scenarios)

Section 1: Architecture & System Design

Q1 Can you give an elevator pitch / high-level summary of RideNow?

RideNow is a production-grade, real-time on-demand ride-hailing and fleet management platform modeled after Rapido and Uber. It features a TypeScript/Node.js backend with PostgreSQL and Prisma ORM, Socket.IO WebSockets, a React 18 customer web app, a React Native driver mobile app, and an 11-module admin operations portal. Key engineering accomplishments include an autonomous proximity-based driver dispatch engine with strict vehicle matching, atomic concurrency locking to eliminate race conditions, and segregated payment pre-authorization holds vs cash-at-completion lifecycles.

Q2 Why did you choose PostgreSQL + Prisma over MongoDB for this project?

- **ACID Transactions:** Ride booking and driver acceptance require atomic multi-table updates. If a driver accepts a ride, updating the ride status, driver duty status, and status history must either all succeed or all roll back together.
- **Relational Integrity:** Relational foreign keys prevent corrupted data (e.g. deleting a user automatically cascades or cleans up related driver/customer entities cleanly).
- **Complex Multi-Condition Queries:** Finding online, approved drivers with a specific vehicle type who haven't declined a ride is vastly more efficient in relational databases using composite indexes.

Q3 How does real-time communication work in RideNow?

We use **Socket.IO** with a hierarchical room architecture:

- **Role Rooms:** `role:DRIVER` is used to broadcast new available trips to online drivers.
- **Entity Rooms:** `ride:{rideId}` and `user:{userId}` are used for targeted events (e.g. sending an incoming ride pop-up directly to the closest captain, or sending status updates to the passenger).
- **Resilient Polling Fallback:** If a mobile driver's network drops packets, the client uses intelligent short-polling (every 3–5 seconds) to ensure synchronization without locking the UI.

Q4 How would you scale RideNow to 1 Million active rides per day?

- **Stateless API Clustering:** Scale Node.js instances horizontally behind an Application Load Balancer / Nginx.
- **Redis Socket Adapter:** Use `@socket.io/redis-adapter` so WebSocket events broadcast across multiple server instances.
- **Geospatial Caching:** Use Redis Geohashing (`GEOADD` , `GEOADIUS`) or PostgreSQL PostGIS for sub-millisecond proximity queries across 100,000+ online drivers.
- **Distributed Message Queues:** Move dispatch iterations and push notifications to background workers using BullMQ / RabbitMQ.

Section 2: Concurrency, Race Conditions & Database Transactions

Q5 How do you prevent two drivers from accepting the same ride simultaneously?

We use an **atomic conditional database update** inside `prisma.$transaction` :

```
tx.ride.updateMany({ where: { id: rideId, status: 'SEARCHING_DRIVER' },  
data: { status: 'DRIVER_ASSIGNED', driverId } })
```

PostgreSQL row-level locking ensures that Driver A's write updates 1 row (count = 1). When Driver B's request runs a millisecond later, the status is already `DRIVER_ASSIGNED` , so 0 rows match (count = 0). The backend catches this, throws `CONCURRENT_MODIFICATION_DETECTED` , and returns HTTP 409 Conflict to Driver B.

Q6 How do you guarantee that a driver is never assigned to two active rides at once?

- **Query Guard:** In `DispatchService` , drivers with ongoing active rides (`status IN [DRIVER_ASSIGNED, DRIVER_ARRIVING, DRIVER_ARRIVED, RIDE_STARTED]`) are strictly excluded from dispatch queries.
- **Atomic Duty Switch:** When a driver accepts a ride, their duty state is set to `BUSY` in the same database transaction.
- **API Guard:** If a driver calls the accept endpoint while their status is `BUSY` , the API rejects the request with HTTP 400 `DRIVER_BUSY` .

Q7 What happens when a driver declines a ride request?

1. The driver calls `POST /api/driver/rides/:id/decline` .
2. The backend upserts a record in the `RideDeclinedDriver` table (`{ rideId, driverId, reason }`).
3. The request is dismissed from that driver's mobile screen.
4. The backend triggers `DispatchService.dispatchRide(rideId)` , which excludes all declined driver IDs and dispatches the request to the next nearest eligible captain.

Section 3: Payments & Financial Safety

Q8 Why did you separate Payment Status from Ride Status?

In a ride-hailing app, money and physical vehicle movement represent two distinct lifecycles:

- **Ride Status:** REQUESTED → SEARCHING_DRIVER → DRIVER_ASSIGNED → RIDE_STARTED → RIDE_COMPLETED .
- **Payment Status:** PENDING → AUTHORIZED → COMPLETED → REFUNDED .
- **Cash Clarity:** For Cash rides, `Ride.status = SEARCHING_DRIVER` while `Payment.status = PENDING` (labeled "*Cash: Pay directly to captain at destination*"). This prevents confusing passengers with false payment errors while clearly tracking receivables.

Q9 How does the Stripe Online Payment Pre-authorization hold work?

We use a **Two-Step Capture** workflow:

1. **Hold:** When booking with a Card, we call Stripe's `paymentIntents.create` with `capture_method: 'manual'` . This places an authorization hold on the customer's card without charging them immediately.
2. **Dispatch Trigger:** Only once the hold is confirmed (status: `requires_capture`) does the ride transition to `SEARCHING_DRIVER` to start matching captains.
3. **Capture:** When the trip reaches `RIDE_COMPLETED` , the backend captures the held funds. If cancelled before driver assignment, the hold is voided with zero transaction fees.

Section 4: Security, Authentication & CORS

Q10 What was the CORS Preflight issue and how did you resolve it?

Problem: When the Admin Portal (port 5174) or Customer Web (port 5173) sent `PATCH / POST` requests with custom Authorization headers to the Backend (port 3000), browsers sent an initial `OPTIONS` preflight request. The Express auth middleware intercepted the `OPTIONS` request and returned 401 Unauthorized because browsers do not attach Bearer tokens to preflights.

Solution: In `auth.middleware.ts`, we added `if (req.method === 'OPTIONS')` `return next();` and registered a universal CORS preflight handler returning 200/204 with explicit `Access-Control-Allow-Methods` and `Access-Control-Allow-Origin`.